Software Project Management – IS212

Test Case Design

Announcements

Announcements – Project Instructions

- The **Project Instructions** are out – please download from eLearn!

- Start figuring out your team's process
  - *Which tools will support you? How will you conduct your meetings?*
  - *How long will your sprints be? How will you document your process?*

- Need any clarifications?
  - *Clarifications on project instructions —> **#help-is212** on Slack*
  - *Clarifications on customer requirements —> via your section's customer briefings only*

- Plan to **start** your first sprint by Week 5

- Plan to **end** your final sprint by Friday Week 12 **(at the latest)**
  - *All deliverables are due **Friday Week 12 @ 23:59***

Announcements – Week 5 Quiz

- **Week 3 Quiz results are out –** *please ask us if any doubts*
  - *Of the 4 quizzes, only your top 3 quizzes retained*
- Our second quiz will take place at the start of your Week 5 class
  - *Starts promptly at the beginning of class!*
  - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
- Short (~10 mins); covers the Weeks 3–4 official materials
  - *MCQ-style questions (no written)*

Recap

Week 3 Recap – Agile Estimation

- What's the difference between an estimate and estimation?
- Why do agile teams prefer **relative** estimation techniques?
- What are story points?
- How can we estimate a product backlog?
- What is a team's velocity and how do we measure it?
- What's the purpose of a burndown chart?





Week 3 Recap – Scrum Meetings

start of a sprint

Sprint Planning Meeting

during a sprint

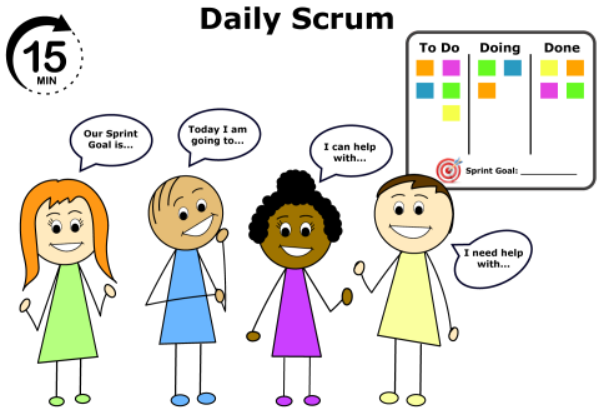
Daily Scrum Meeting

end of a sprint

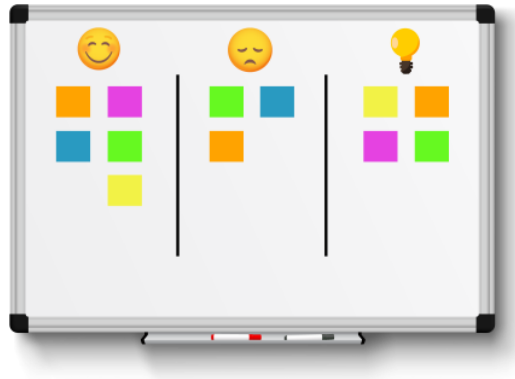
Sprint Review Meeting

Sprint Retrospective









Today: Agile Testing & Test Case Design

Objectives

On completing this class, you will be able to:

- Explain how testing fits into an agile software development process
- Derive test cases from the acceptance criteria of user stories
- Identify positive, negative, and boundary test cases for different features
- Integrate test case design into your scrum process

Outline

- ***Software testing*** **–** traditional vs. agile testing; agile testing quadrant
- ***Test case design*** **–** 5-step approach to identifying test cases
- ***Test cases in scrum*** **–** when to write; Definition of Done; where to store

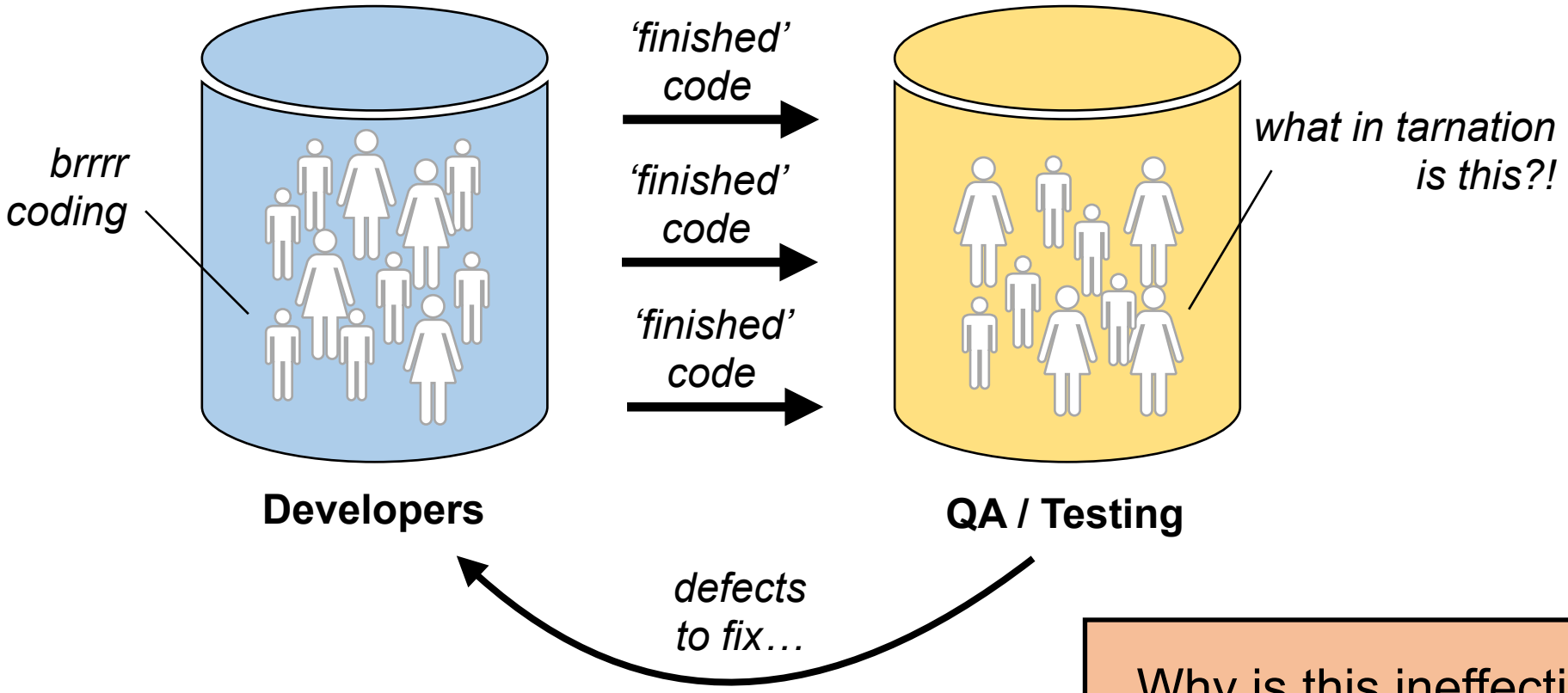
SMU Classification: Restricted

Software Testing

ensuring* that software satisfies *expectations

How **Not** to do Testing

- In the past, developers and testers worked in their own silos

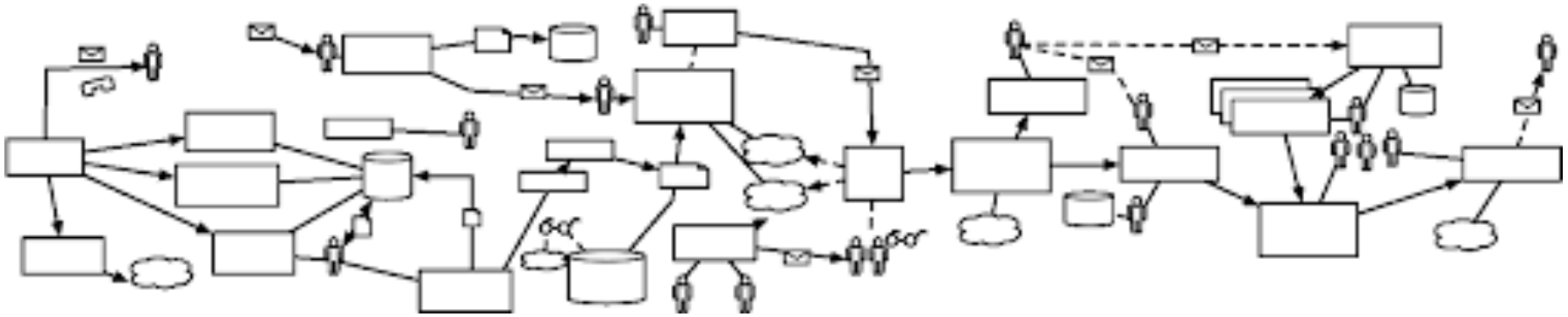


Why is this ineffective?

How **Not** to Integrate New Code

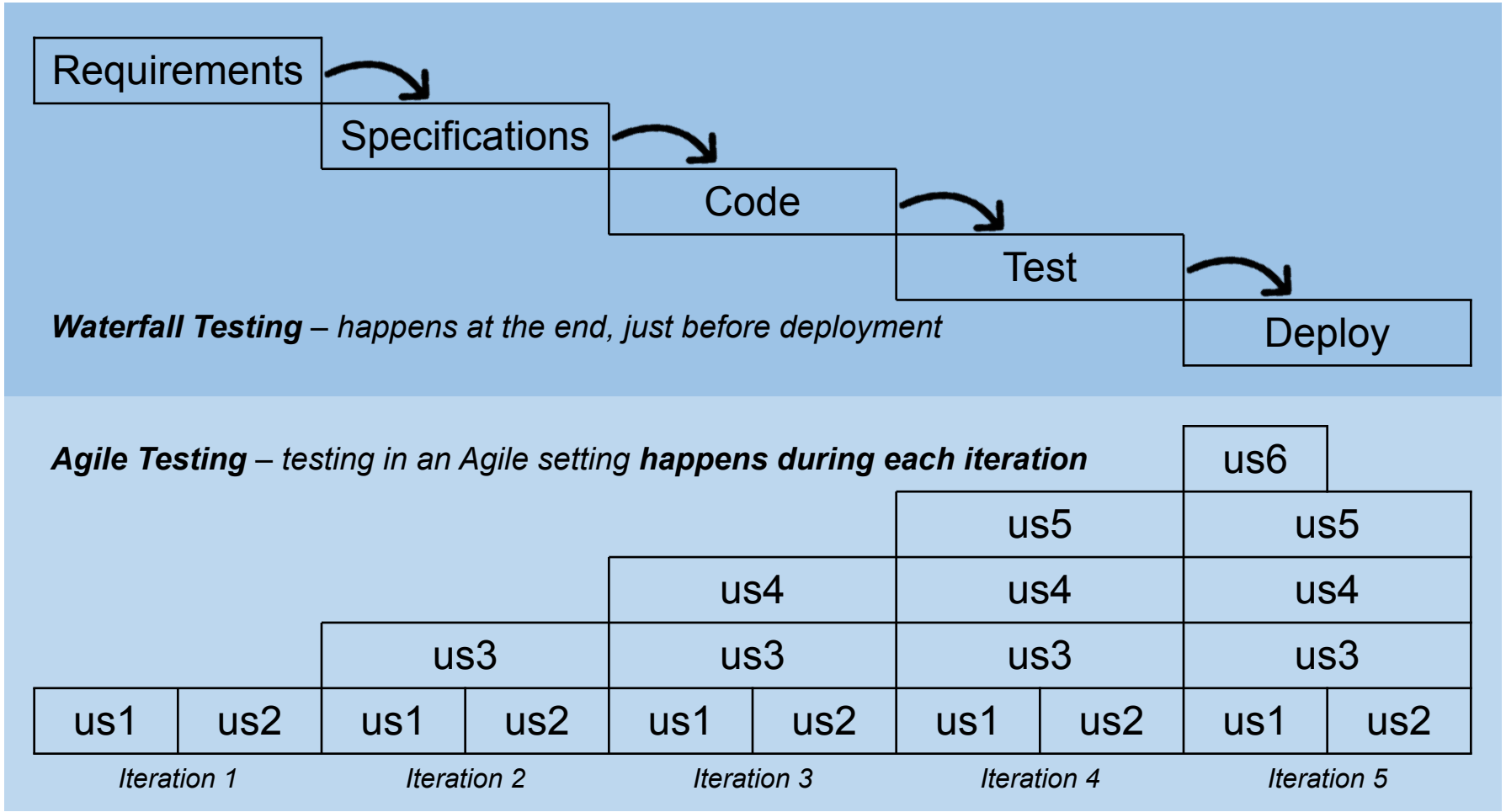
- ***Worst-case scenario –*** large, complex organisation; tightly coupled monolith; scarce integration testing; many approval processes

- What a deployment lead time of three months can look like:



- After merging all the changes, the monolith no longer builds
  - Heroics and multiple investigations required
  - Who broke it? How can it be fixed? Delays for the customer? $$s?

What do Agile Practitioners do Differently?



What do Agile Practitioners do Differently?

- Testing is a core activity in **every iteration** (i.e. sprint)

- Testers and developers (possibly the same people) work closely together in cross-functional teams (i.e. the scrum team)
  - *Break down silos*
  - *Build a shared understanding*
  - *Frequent and regular feedback (e.g. sprint reviews)*

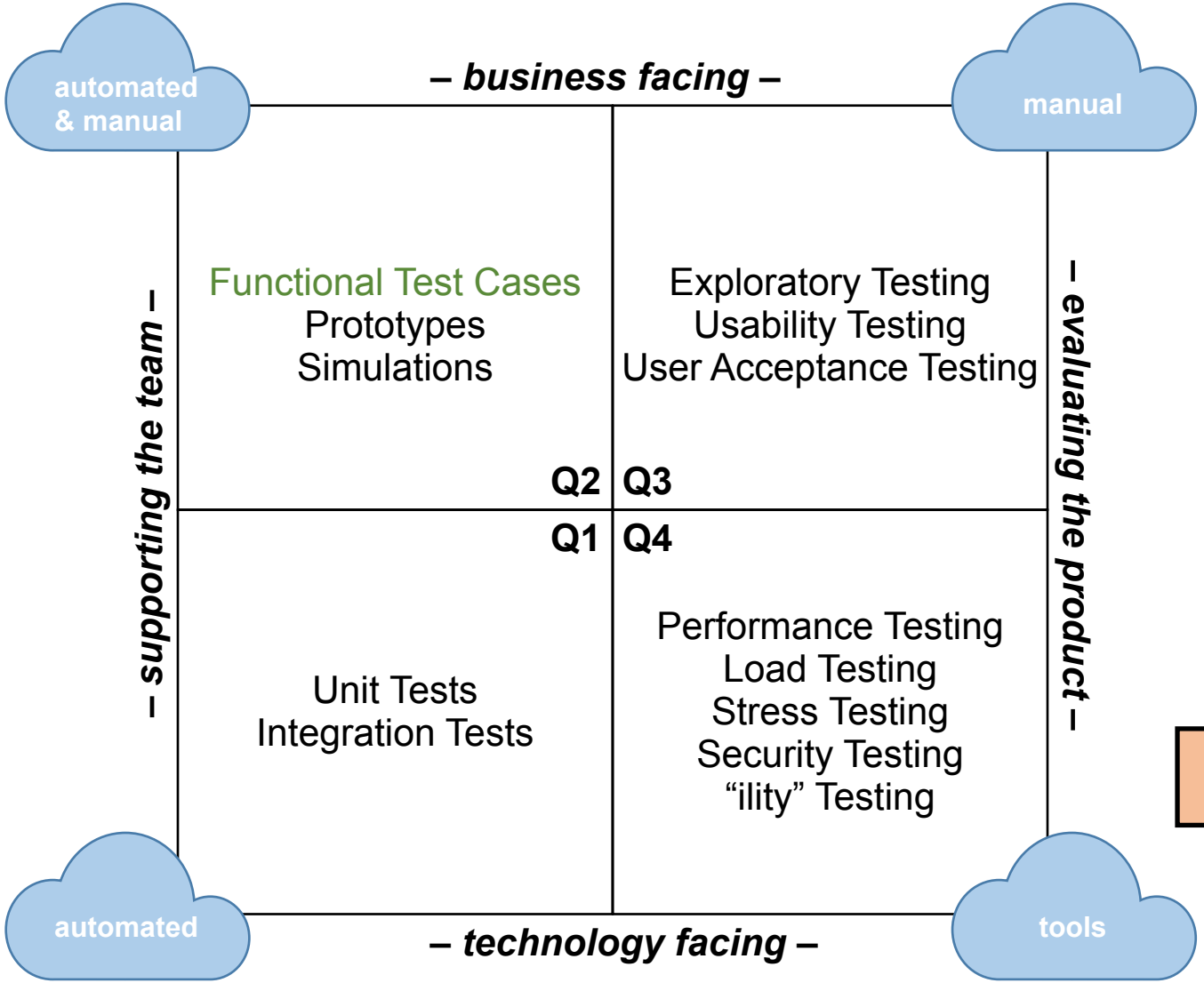


- Developers frequently integrate code into a shared repository
  - *Frequent, smaller changes → diagnose issues earlier & easier fault localisation*

- Automation of tests **(week 6)** to support the practices of continuous integration **(week 7)** and refactoring **(week 11)**

In **what ways** can a system be tested?

Agile Testing Quadrant



Which tests matter?

Which Tests Matter?

- Minimally, we need (functional) test cases for your user stories
  - *i.e.* ***detailed*** *description of steps/conditions to verify that a function works as intended*
  - *Shared understanding of the expected behaviour of the feature*
  - *Facilitates user story acceptance by the Product Owner (and/or end users)*

- Automated unit/integration tests for user stories are also valuable
  - *Helps ensure that new code changes can be integrated 'safely'*
  - *Facilitates regression testing – maintain integrity of the software over time*

- Beyond this… **what does the customer want?**
  - *Is there a performance / scalability / accessibility / … requirement?*
  - *Is a potential production outage a mild inconvenience, or a serious threat ($$s) to the business?*

Example – Stress Testing in Production

- ***Netflix –*** introduced tension in production to test the "resilience" of their recovery systems
  - E.g. Chaos Monkey – randomly kills processes and compute servers in production
  - (Why production and not a test environment?)
  - "Chaos engineering"



- Survived a massive AWS outage in 2015:
  - https://www.techrepublic.com/article/aws-outage-how-netflix-weathered-the-storm-by-preparing-for-the-worst/

SMU Classification: Restricted

Test Cases

Recap – User Story Card

Title *Good short title*	**Priority** *Order of implementation*	**Estimate** *Story point value*
User Story *Simple description (no details) of customer/user need* ***(conversation starter)*** *As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome>*		
Acceptance Criteria *List of conditions fulfilled on completion of implementation*		



- Boundaries of the story
- Use to check if the story ‘done’

Recap – User Story Card

Title *Accept Credit Card Payments*	**Priority** …	**Estimate** …
User Story *"As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently."*		
Acceptance Criteria Ensure that the customer: • Can input a valid card number, expiration date, CVV, and cardholder name • Can only pay with Visa, Mastercard, or American Express • Is requested to provide an OTP • Is immediately informed of payment success or failure • Has a secure SSL/TLS connection		

Can we translate these to tests?

Test Cases

- A test case is a **specific, detailed** description of the steps and conditions required to determine whether a particular function or feature works as intended

- Broadly, they exist to confirm that the acceptance criteria (ACs) have been met

- Unlike ACs, functional test cases consider the **specific outputs** of the application **given specific inputs**
  - *Test cases must be clear and unambiguous enough that anyone can follow*
  - *E.g. "log in to the app"* ❌ *vs. "log in with username X and password Y"* ✅

Test Case Template

Item	Description	Example
Test Case ID	Unique ID of the test case	TC-001
Test Scenario	Succinct summary/objective of the test case	Check login with valid credentials
Pre-conditions	Conditions to be fulfilled prior to executing the test, e.g., database must contain right test data	Run **tc1_setup.sql** on testDB
Test Steps	Step-by-step procedure to execute the test; this is what the tester must follow	1. Navigate to https://abc.com/login 2. Enter UserId and Password 3. Click Submit
Test Data	The specific inputs used to conduct the test	UserId=user99; Password=pass99
Expected Result	Expected result from the test	User logs in successfully; home page shown
Actual Result	Actual result from the test (to be filled post-test)	User logs in successfully; home page shown
Pass/Fail/Not Executed/ Blocked	Status of the test	Pass
Remarks	Any comments on the test	NA
Created By*	Author of test case	John T.
Date of Creation*	Date test case created	22/02/2024
Executed By*	Person who ran test case	John T.
Date of Execution*	Date test case was run	22/03/2024

* test case metadata

Why is it important to reset the test database?

Purpose of the SQL file is to prepopulate the tables

Deriving Test Cases from ACs

- Notice the similarity between scenario-based ACs and test cases

Acceptance Criterion	Test Case
Given… When… Then…	Pre-conditions… Test Steps/Test Data… Expected Result…

- While not a compulsory step, it can be a helpful exercise to first translate checklist-based ACs into scenario-based ACs in order to derive test cases
  - *Recall that a checklist-based AC can be refined into multiple scenario-based ACs…*
  - *…and thus multiple different functional test cases*

Example – ACs for Valid Passwords

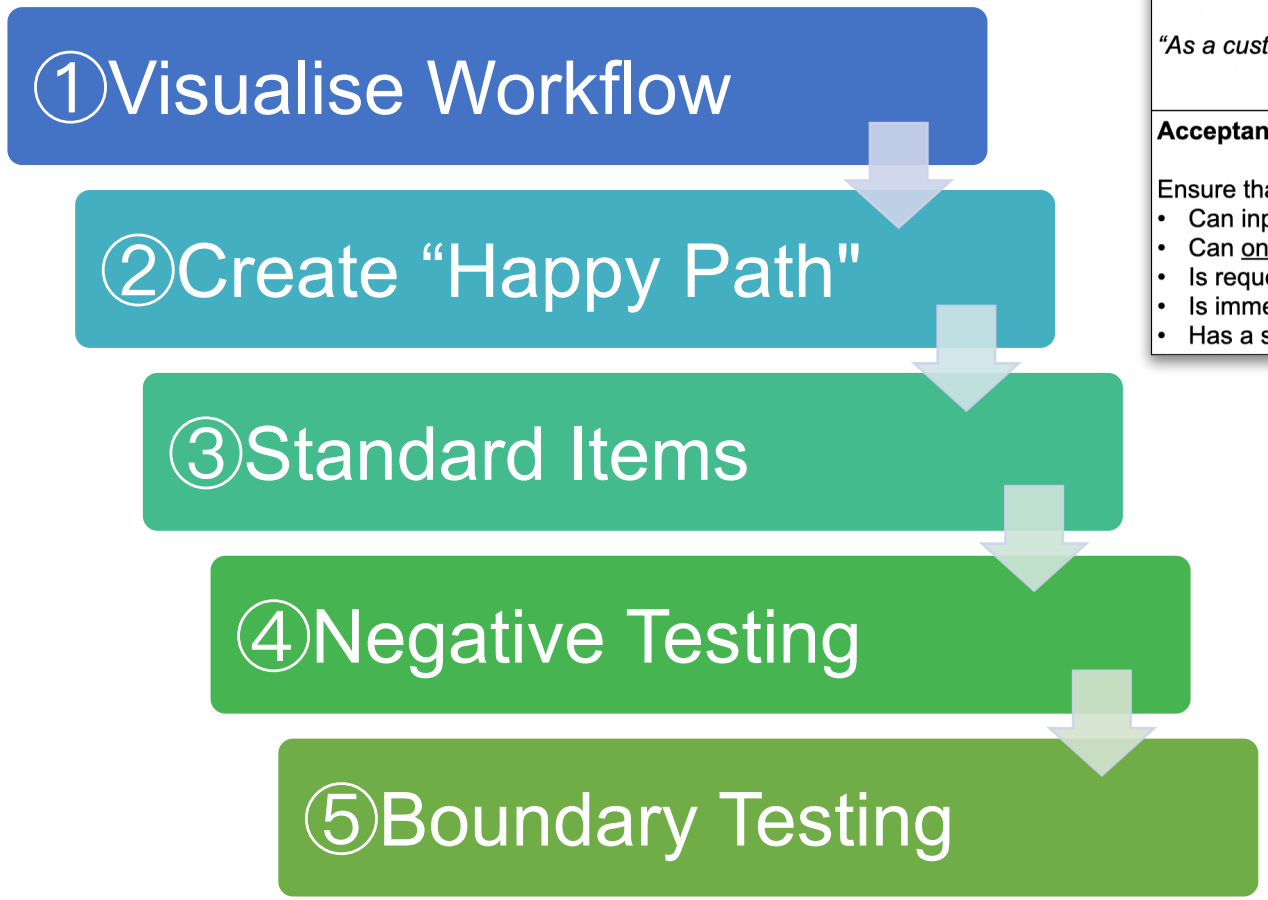
- To illustrate, consider the many scenarios (and thus test cases) associated with entering a "valid" password

***How many** test cases do we need?*

STRONG PASSWORDS ACCEPTANCE CRITERIA

Data	Expected result	Expected message
Aa9ab$$	Fail	Too short
AAbbCC11	Fail	No special characters
$$$bbb111	Fail	No upper case
AAA%%%1111	Fail	No lower case
AAA%%%%bbbbb	Fail	No numbers
AAAA(((bbbb	Fail	Braket is not a special character
BBBBB}}}hhhh	Fail	Brace is not a special character
256 characters input	Fail	Max password length is 255
IsThis$AGood11	Pass	
IsThis~Good11	Pass	

Identifying Test Cases



User Story

“As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently.”

Acceptance Criteria

Ensure that the customer:

- Can input a valid card number, expiration date, CVV, and cardholder name
- Can only pay with Visa, Mastercard, or American Express
- Is requested to provide an OTP
- Is immediately informed of payment success or failure
- Has a secure SSL/TLS connection

①Visualise Workflow

- Put yourself in the end user's shoes 👞👟👠
- Visualise what they see and how they might behave, and derive test cases for these different behaviours
- What happens when they…
  - *Click "Order History"?*
  - *Click "Add to Cart"?*
  - *Write their name in the card number field?*

Can we propose test cases before the feature is built?

- If you're not familiar with the feature, converse with the team, or navigate through other existing features to get a 'feel'

②Create “Happy Path"

- What is the path through a feature that encounters no errors or exceptions? (Also known as the “positive test case”.)

- For our user story, this would be a test case in which the user successfully pays for a (non-empty) cart with a credit card
  1. Select cart item(s)
  2. Click checkout
  3. Select card payment
  4. Card payment screen is shown
  5. Enter card details
  6. Click submit
  7. Purchase confirmation screen is shown

- Test cases derived from the happy path are usually the first to be executed when validating a feature **(Why?)**

③Standard Items

- Are there “standard objects” that should be present in every feature? For example:
  - Copyright footer
  - Terms & Conditions link
  - Standard headers
  - Standard menu items (e.g. breadcrumbs)

- These can be easily missed, and some may need test cases

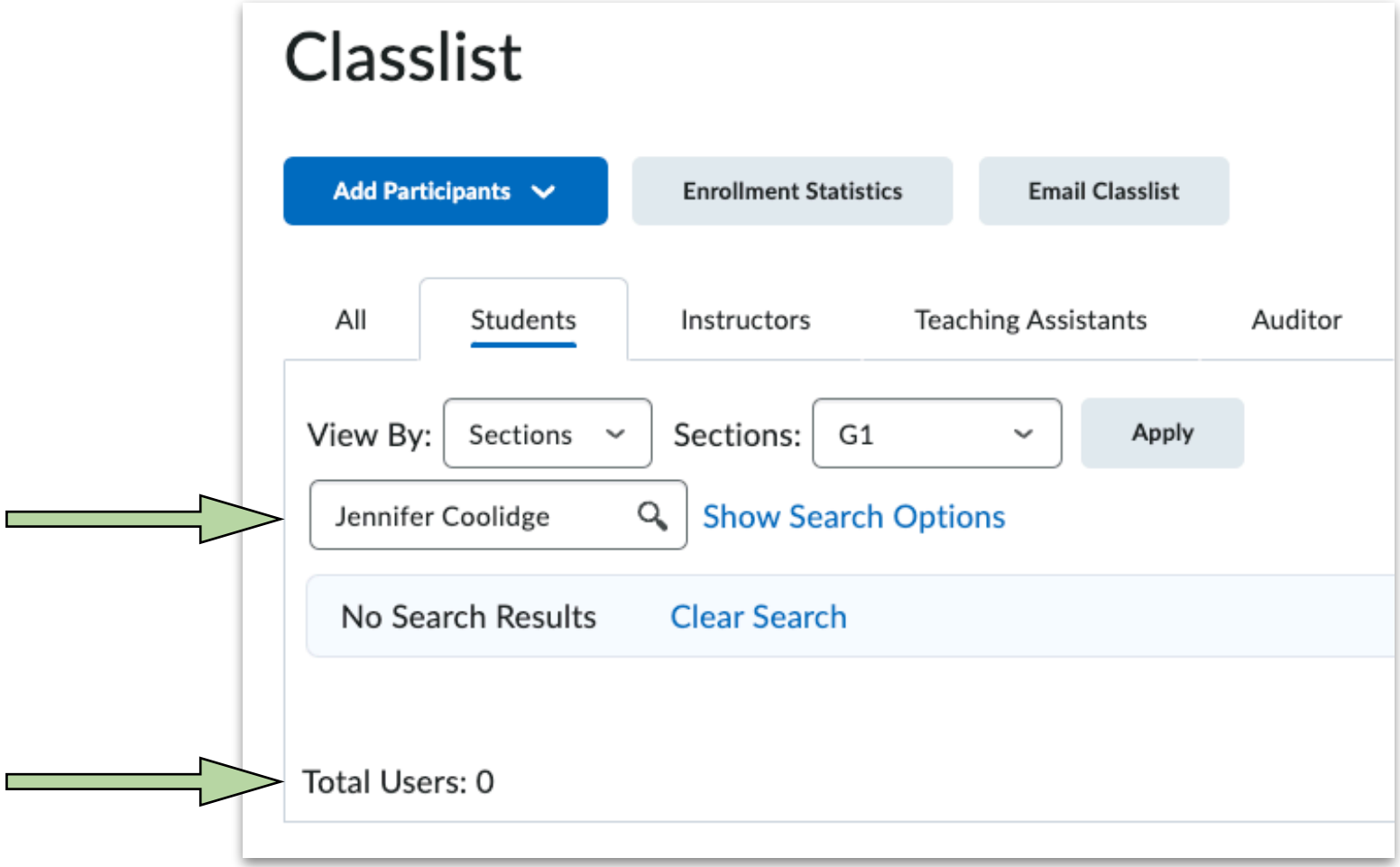
*Should we write test cases for the **same** UI guidelines for **every** feature?*

④Negative Testing

- We can't expect the happy path to be the only path

- Sometimes, users enter erroneous data. Sometimes, business exceptions occur. And sometimes, systems are unavailable. E.g.:
  - *User enters their name in the credit card field*
  - *User enters an (unsupported) Discover card number*
  - *A successfully ordered item becomes out of stock before dispatch*
  - *The payment vendor (e.g. Stripe) is unavailable*

- We need test cases to verify that these "sad" events are handled correctly, e.g. through appropriate error messages



Aside: *is this a negative test case?*



⑤Boundary Testing

- Does the scenario involve ranges of inputs?

- Are there **boundary values** above/below which the outcome changes? E.g., a $20 minimum order:
  - A $19.99 order ❌
  - A $20.00 order ✅
  - A $20.01 order ✅

"Just inside, and just outside too"

- Bugs / logical errors are surprisingly common around boundary values – **why?**



*What's the **boundary** we need to test?*

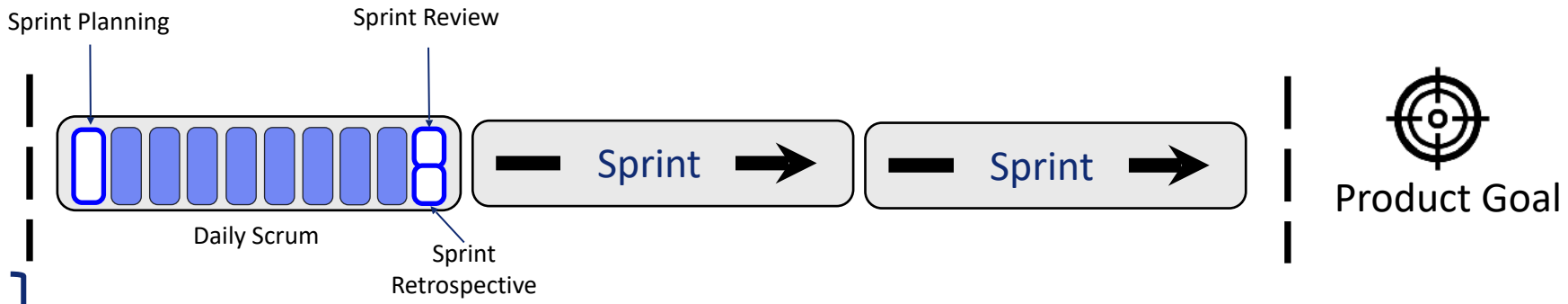
Use mock data to test if there are no user data

Item	Content	
Test Case ID	TC-001	
Test Scenario	Test that the visa credit card payment is successful.	
Pre-conditions-setup **Db instance to what uw the user to see**	Testcase_TC001_visa_credit_card.sql (populate my shopping card and customer delivery data) if u use mongodb then what is the script uw to execute to pump into mongodb	
Test Steps	1. visit http://shopingtest.com.sg/login 2. Login as user 3.click "view checkout" 4. Click "Confirm" 5. Enter payment details 6. Click pay	
Test Data	Userid:shopper1 password:abc123 Credit card name: John lim Credit card no "420000000" Cvv=123 Expiry date: 30 Jan 2029 OTP:123456	
Expected Result	1. When you clicks on "view checkout" you should see products (P1,"water bottle",qty 5, total amount…) total amount is s$120. 2. Afer u click pau u will see the confirmation of success on the screen	If is a fail test case then credit card number will be different (invalid one) Checks can be done locally within the system doesn't need to send over => then in expected result. Need add " after step 7, system will prompt that the credit card is not a valid visa card number. You will see "credit cared is not a a valid visa card number"

SMU Classification: Restricted

Test Cases in Scrum

When to Write Test Cases?



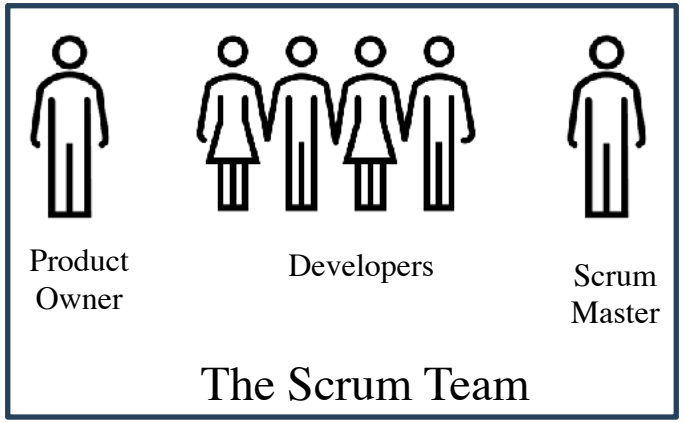
Product Backlog Items

- User Story 1
- User Story 2
- User Story 3
- User Story 4
- User Story 5
- Epic 1

Sprint Backlog

Sprint Goal: To complete …

	To Do	In Progress	Done
User Story 1	Write the front-end code; Write the back-end code; Do the functional testing; Update System Docs		
User Story 2	Task; Task; Task		



① As User Story is written, Acceptance Criteria are created for which test cases will be written

② As Conversations happen, additional test cases might be identified

③ In the sprint planning meeting, the team might go through User Stories and add additional Acceptance Criteria, if any

④ Team might discover additional test cases before or after coding

***TLDR:** test cases can be written any time until a user story is marked done*

Definition of Done (DoD)

- This begs the question: **when exactly** is a user story "done"?

- This is agreed by the scrum team and stakeholders in a specification called the Definition of Done (DoD), e.g. a checklist of items such as:
  - **Test cases pass** (i.e. ACs met)
  - Unit tests pass
  - Code reviewed
  - **Non-functional requirements** satisfied
  - UI guidelines satisfied
  - Product owner accepts
  - …

Can you think of some examples?

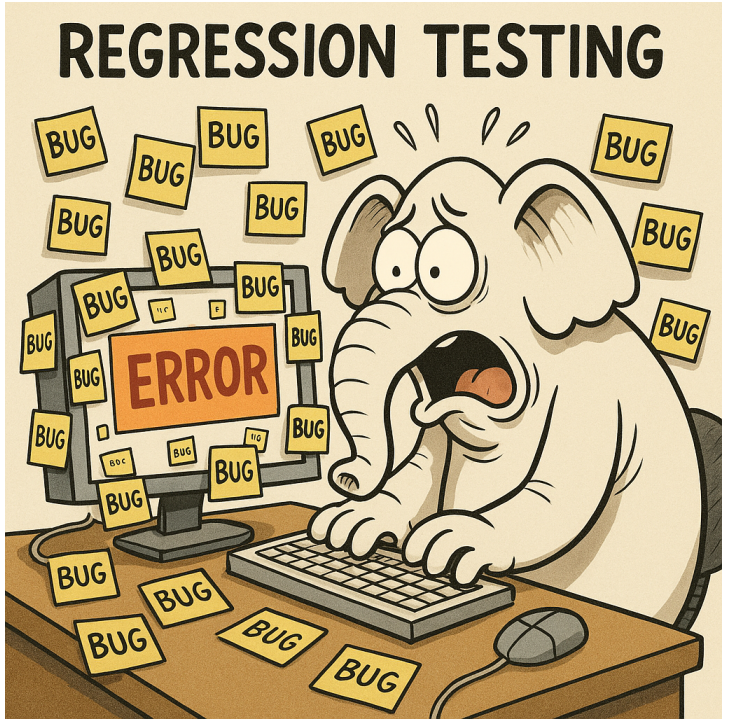
- If a user story doesn't meet the DoD, then it returns to the product backlog

Regression Testing

- Regression testing involves re-running test cases from previous sprints to verify that existing code still functions as expected

- Why does it matter in scrum?
  - *Frequent code changes may lead to unintended side effects*

- How should it be integrated into scrum?
  - *Can be part of the **DoD checklist***
  - *Can be automated in continuous integration workflows **(week 7)***



Where to Store Test Cases?

- There are many different approaches for storing / managing test cases, and you are encouraged to research and choose one that suits your team's working style

- For instance, a team wiki (e.g. on Confluence) or spreadsheet (on Google Sheets)
  - *Do research online! There are templates* 😊

- Alternatively, if you are using project management software (e.g. Jira), see if there are plug-ins available.
  - E.g. the Zephyr plug-in for Jira

Test Case Traceability

- Test case traceability is the ability to link each test case back to its corresponding requirement or user story

- Helps ensure that:
  - *All user stories are tested*
  - *All tests map back to real requirements (user stories)*
  - *Code changes are justified and tested*

- Can use a traceability matrix to track these linkages, e.g.

User Story	Acceptance Criteria	Test Case ID	Related Code Module
"As a customer, I want to pay by credit card…"	Only Visa/MC/Amex allowed	TC-003	`payments/ validate_card.py`

Software Project Management – IS212

Distributed Version Control (Git)

Today: Distributed Version Control (Git)

Objectives

On completing this module, you will be able to:

- Identify the benefits of distributed version control for software developers
- Explain the fundamental concepts and mechanisms of Git
- Collaborate on software projects using basic Git commands
- Resolve merge conflicts using Git



Topics

- ***Git*** – rationale, fundamental concepts, basic commands, conflicts, tips
- ***Lab*** – hands-on practice with Git, GitHub, and GitHub Desktop

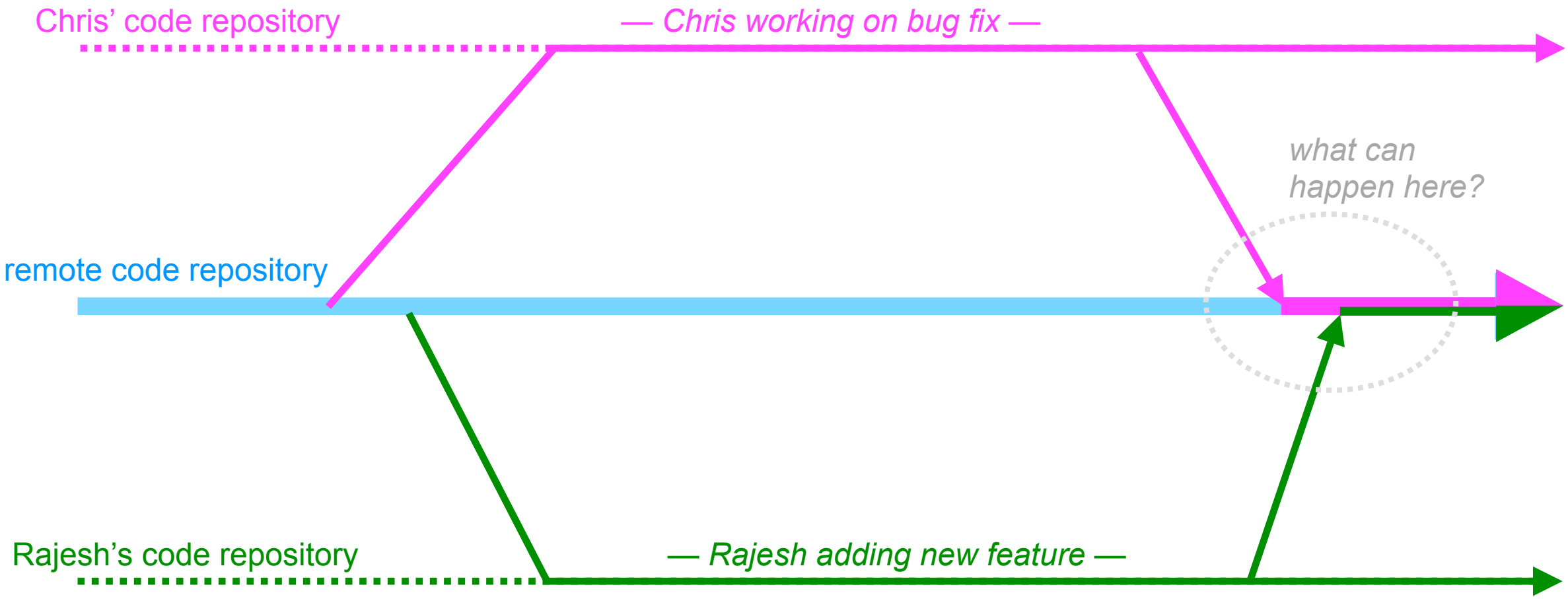
Why version control?





in other words – *why are general-purpose collaboration tools inadequate for software development?*

Example Scenario



Git

- Git is a distributed version control system
- Provides mechanisms for sharing changes to source code and solving merge conflicts
- Provides full control over when and what you 'publish' to the central code repository
  - *Experiment without disrupting the 'main' codebase*
- Systematically tracks history of changes
  - *Allows you to revert to an earlier version*



Linus Torvalds

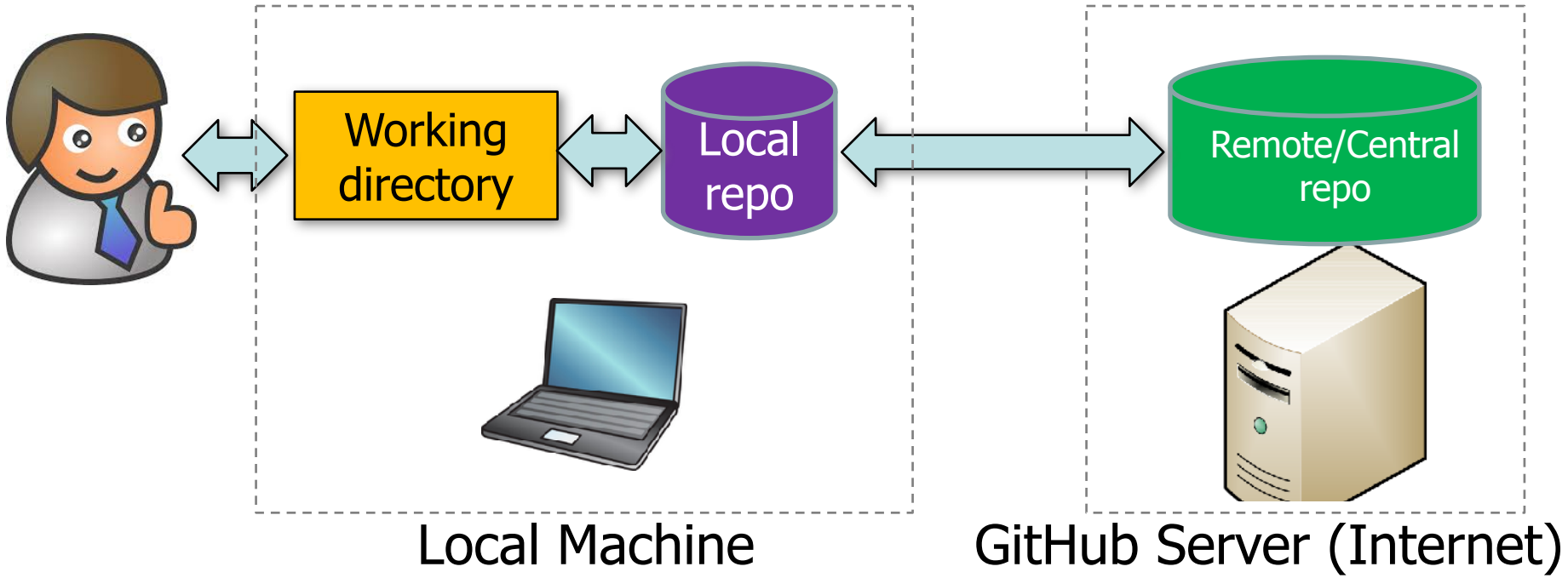
Aside: what does “git” mean?

“Git” which means "unpleasant or contemptible person" in British slang 🇬🇧

*Torvalds: “**I’m an egotistical b*st*rd, and I name all my projects after myself.**”*

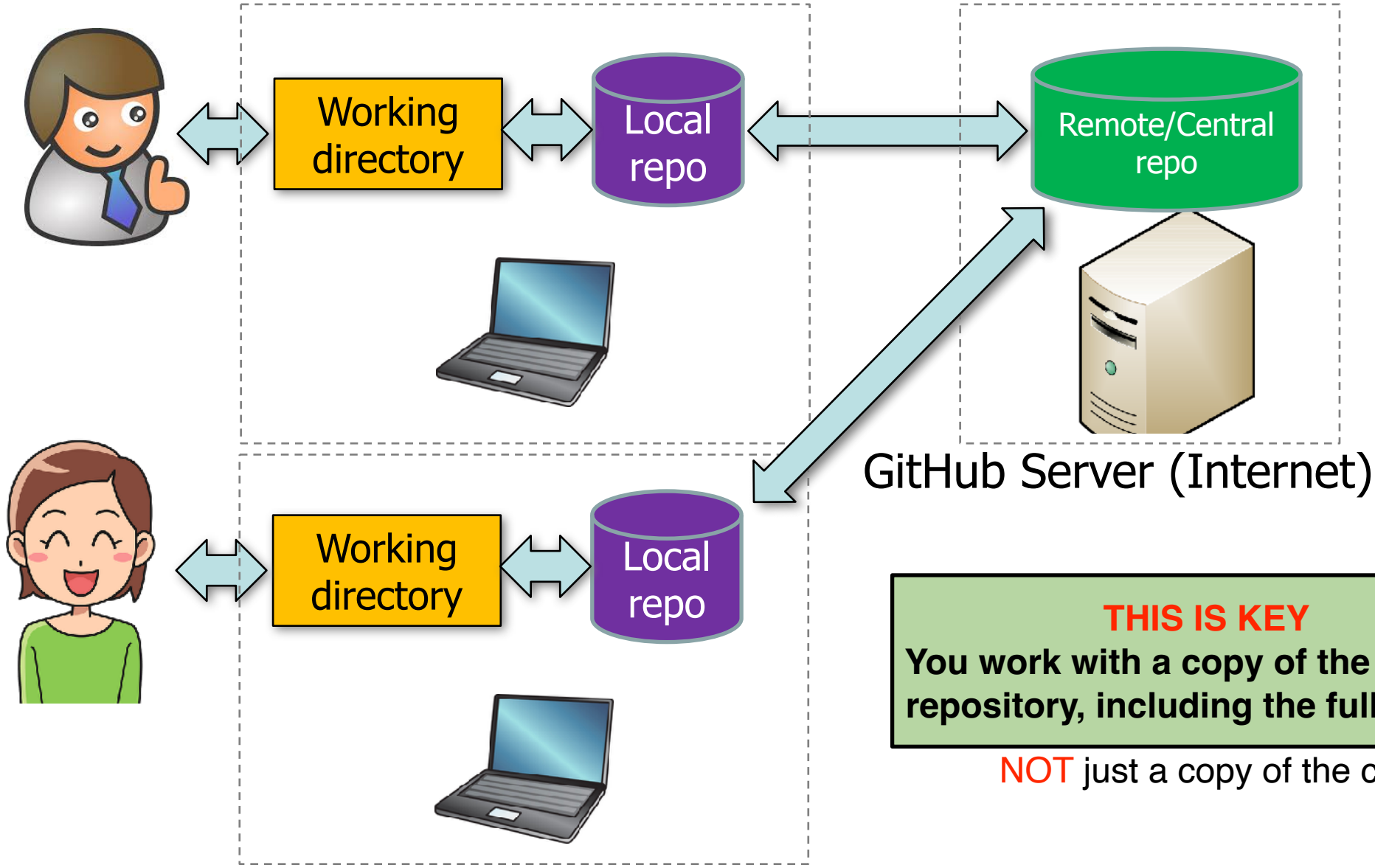


Git – key entities



…or GitLab server, Bitbucket server, …

Git – *it's distributed!*

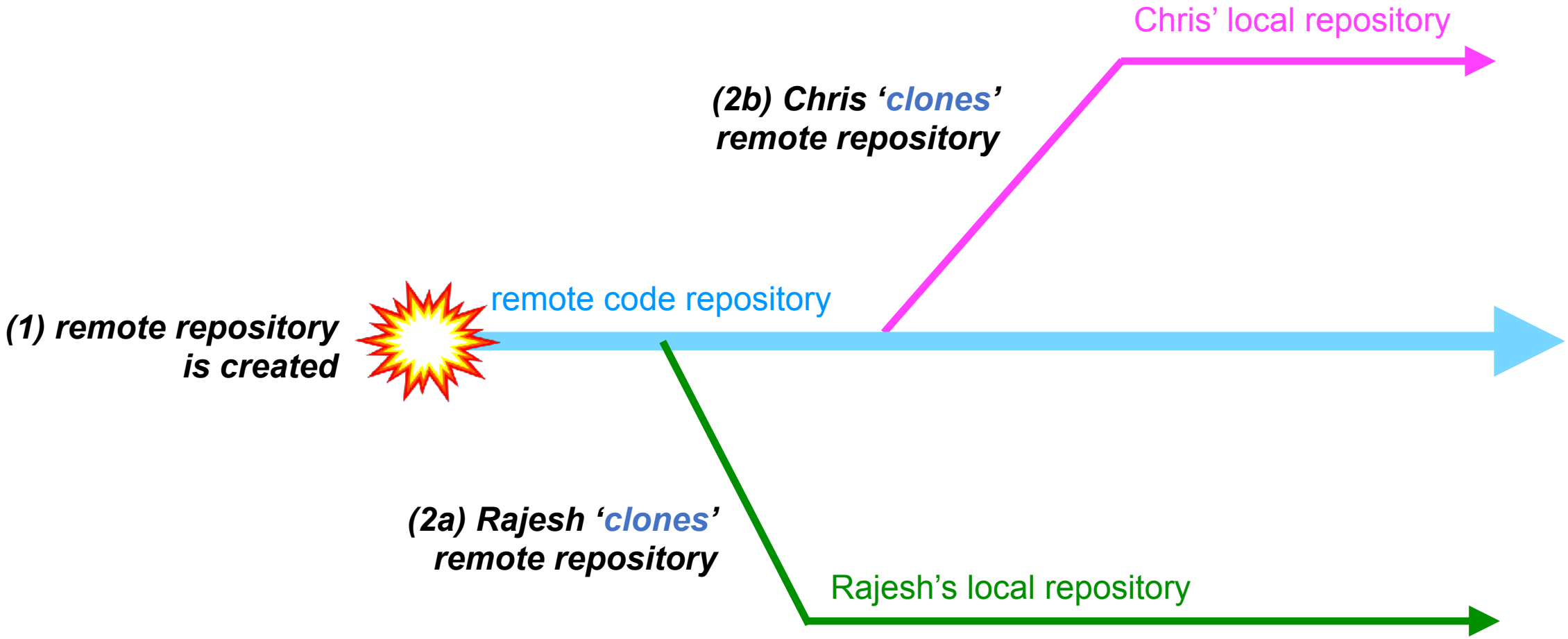


THIS IS KEY

You work with a copy of the *whole* repository, including the full history

NOT just a copy of the code

Key Steps – Setting up



Key Steps – Setting up

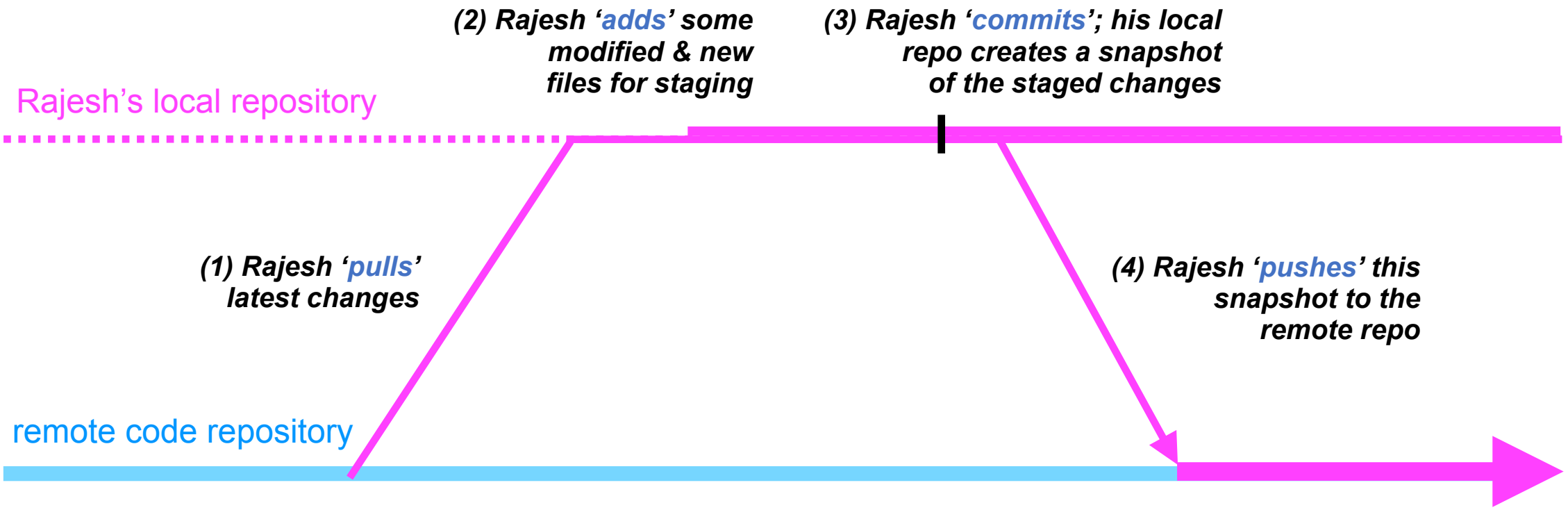


- Create a new remote/central repository
- Host on external server, e.g. GitHub

git clone

- Get a full local copy of the central repository
- i.e. a history of every version of every file
- Executed once, at the start of (your participation in) the project
- CLI: git clone *<url>*

Key Steps – Making changes



Key Steps – Making changes

git pull

- Retrieves up-to-date copy of remote repo
- Local repository updated to reflect changes
- CLI: git pull

git add

- Marks files to be staged for the next commit
- Can be **new files** and/or existing **modified** ones
- CLI: git add <*file*> or git add <*path*>

Key Steps – Making changes

git commit

- Creates snapshot of project's staged changes
- Recorded in local repo; not yet remote one
- CLI: git commit -m "<*message*>"

git push

- All committed changes in the locally created snapshot are sent to the remote repo
- Your changes can then be pulled by others
- CLI: git push

Demo **–** Basic Git commands

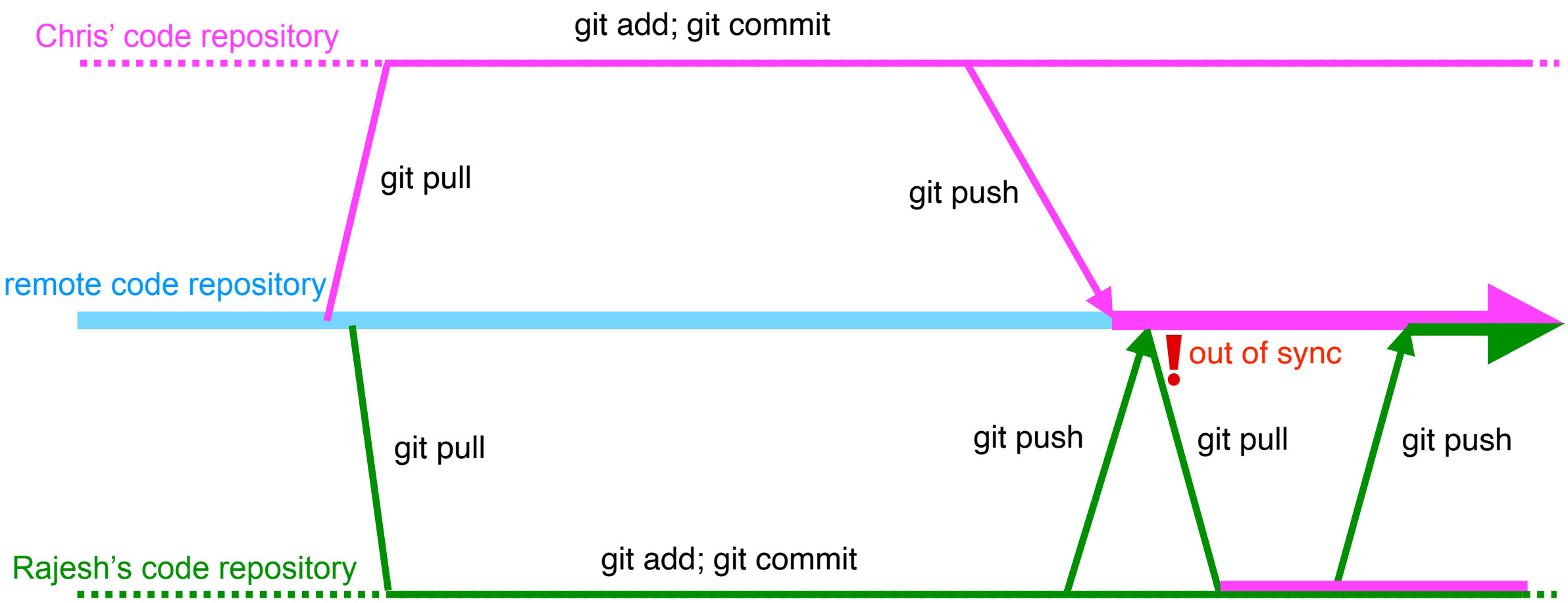
Conflict Management

- Git's conflict management approach is one of the key reasons for its popularity
- Protects against human coordination errors
  - *e.g. two devs changing the same line of code*
- Facilitates safe decentralised editing and efficient code merging
- ***However –*** the absence of conflicts doesn't mean that changes won't break code! **(Solved in Week 7)**



Need to come up with a conflict resolution process

Git Automatically Merges Commits – *if it's possible!*



Fixing Conflicts

- If Git cannot automatically merge in remote commits, you will be asked to resolve them directly in the source code
  - The two versions are marked in the code; replace the markers with the correct code

```
def example_function():
<<<<<<< HEAD
    print("This is my local change.")
=======
    print("This is the remote change from origin.")
>>>>>>> abc1234
```



```
def example_function():
    print("This is my merged change.")
```

- After saving the changes, remember to stage, commit, and push
  - git add, git commit, git push

***Demo* – Causing & Fixing Conflicts**

Advanced Git – Tagging

- Use tagging to provide names to **mark key places** in the git checkin history
  - release 1, end of sprint 2, etc.

- This allows you to search and find these place by name instead of the (random SHA1 hash) git checkin ID

- Create a tag
  - git tag –a “v_1” –m “This is our first release” <optional last bits of commit id>
  - git push origin v_1

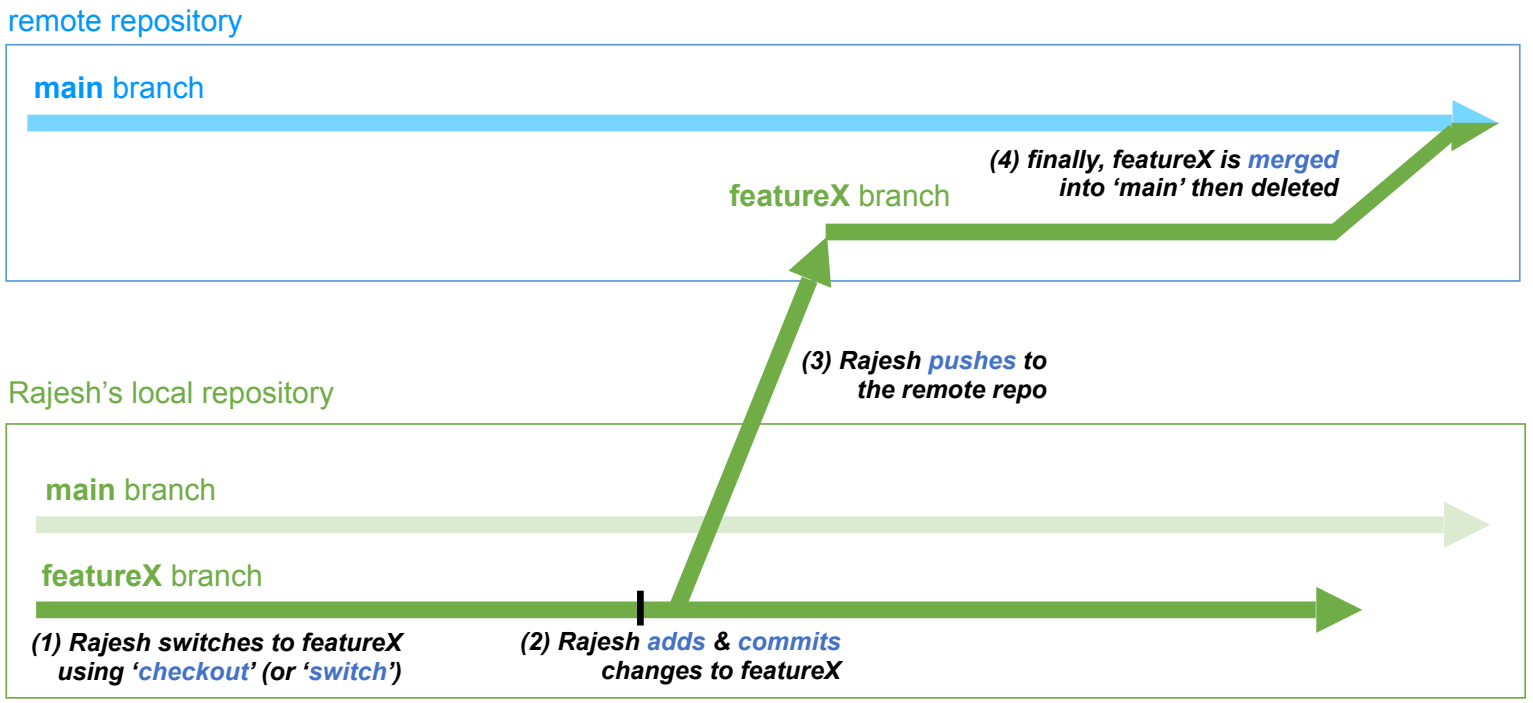
- Using a tag
  - git checkout v_1

- https://git-scm.com/book/en/v2/Git-Basics-Tagging

Advanced Git – Branching

- Git also supports branching, which allows each repository to have multiple diverging versions of the codebase, allowing developers to work in isolation from the changes of others

- To be covered in **Week 7**; in the meantime, try trunk-based development for Sprint #1 😊



Tips – Ignoring files

- Use a .gitignore file to inform Git to ignore certain file extensions

- E.g. log files, .DS_Store (on Mac), …
  - Some good examples: https://github.com/github/gitignore

```
# Compiled class file
*.class

# Log file
*.log
```

```
# Java Package Files
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar
```

Tips – Structuring a Git Repository

- Organise code logically, keeping the root folder uncluttered, e.g.:

  /src → *Source code*
  /tests → *Test cases*
  /docs → *Documentation (README, design docs, etc.)*
  /scripts → *Utility or setup scripts*
  /config → *Configuration files (e.g. .env or YAML files)*

- Include essential metadata, e.g. README.md and LICENSE files

- Separate concerns – don't mix production with test code

Tips – Structuring a Git Repository

- Structure your repository for growth
- Avoid giant files (e.g. main.py) that contain ***ALL*** the logic!
- Even small projects benefit from being modular
  - *i.e. grouping related files into packages and modules logic!*
- Look at existing (well-starred!) open source projects that use your software stack for inspiration on how to organise your project

Tips – Undoing changes

Unmodify a modified file:

Revert a changed file to its most recently committed version
"git checkout <file>"
All changes to the file in the working directory are gone

Remove a file from Git tracking:

First untrack the file: "git rm --cached <file>", then run "git commit"
The file will be removed from Git, but remains on your folder

Undo last commit:

"git revert HEAD"
Create a new commit inverting the commit to be undone
The commit history is kept

Tips – *Careful what you make public*

```
8
9   app = Flask(__name__)
10  app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+mysqlconnector://phriscoskitt:bananaYes!@my.database.com:3306/game'
11  app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
12
```

- Don't commit passwords, API keys, etc.
- If you accidentally do, consider them compromised
- *(Why is deleting them in the next revision NOT enough?)*
- *(How should you pass passwords/keys to the app?)*

Tips – Git Best Practices

- Do write meaningful commit messages
  - *Usually the only thing that is easily seen in GitHub*
  - *Allows everyone to know what was changed without looking at the code*

- Do use Git consistently and correctly
  - *Git provides you benefits only if you actually use it*
  - *Make sure you are working with the right version (pull if needed)*
  - *Even if doing a "throw away" implementation, use Git*
  - *If you like the final result, you can merge it*
  - *If your code stops working, you can revert easily to a working version*

Tips – Git Don’ts

- Don’t commit/push using someone else’s account
  - *Commit logs help determine individual contributions to the project*
- Don’t zip up your files and commit to git
  - *That’s not version control! Check in the source files directly*
- Don’t commit/push IDE-specific utility files
  - *They can lead to conflicts. Use .gitignore*
- Don’t copy/move the .git directory at the top of your local repo
  - *This will confuse git’s version tracking logic*
- Don’t create separate repos to “avoid conflicts”. E.g. front-end, back-end
  - *This is just trying to hide deeper issues..*

References

- Pro Git Book
  - https://www.git-scm.com/book/en/v2
- Git Guide
  - https://nulab.com/learn/software-development/git-tutorial/git-basics/

Exercise & Lab

- **Exercise:**
  - Identify the highest-priority user stories in your product backlog
  - Start writing test cases for them using the 5-step approach

- **Lab:**
  - Need more practice with Git?
  - Optional lab available in eLearn
  - Shows you how to set up a repository and handle conflicts



Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet ☺
- **http://smu.sg/is212-reflections-2025**

